ShopSmart AI

Solution design

1. Problem and solution overview

Customers struggle to find suitable products, compare options, understand shopping policies, and obtain order information. ShopSmart AI provides a responsive shopping interface and one AI assistant that uses catalog data, approved knowledge documents, and customer order information.

The application demonstrates the case-study requirements using sample data, approved sample documents, simulated orders, and mocked delivery events.

Committed scope

Area	Implementation commitment
Shopping	Login, catalog, category hierarchy, search, category/brand/price filters, price/rating sorting, pagination, comparison, wishlist, cart, preferences, and responsive layout.
Recommendations	Interpret intent, extract requirements, apply product constraints, recommend catalog matches, explain reasons, check availability, and clarify incomplete requests.
Knowledge assistant	Upload PDF, Markdown, and JSON; extract, chunk, embed, retrieve, and generate grounded answers with document references.
Order tracking	Retrieve simulated customer orders and mocked shipment events.
Agent coordination	One assistant calls backend services for product search, inventory, order status, and knowledge retrieval.
Explainability and audit	Display reasons and citations; record requests, tool activity, source references, outcomes, and timestamps.

Scope boundaries

- No payment integration.
- No real carrier or external marketplace integrations.
- No stock forecasting, promotion management, or pricing automation.
- No production-scale deployment in the initial implementation.
- Cancellation eligibility is informational; the assistant does not execute cancellations.

2. Assumptions and user journeys

Implementation assumptions

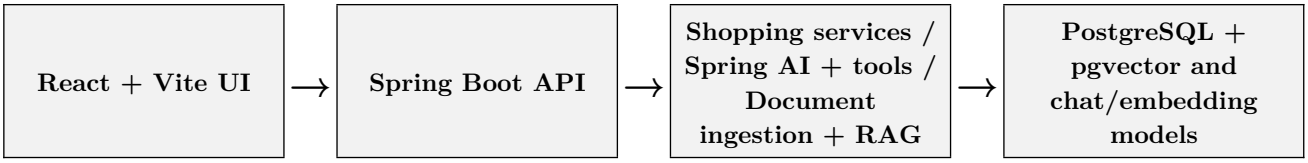
- Seeded demo accounts represent existing customer authentication and identify Basic, Prime, or Premium membership.
- Sample products include the required product fields and specifications for comparison.
- Inventory is centrally managed; an internal warehouse API supplies seeded warehouse data.
- Orders are simulated and delivery events are mocked.
- Approved knowledge documents include version, effective date, category, and region.
- Chat and embedding models are accessible during development.

Required user journeys

Journey	Required behavior
Login	Sign in; view membership and preferences.
Browse and find	Navigate categories; search; filter by category, brand and price; sort by price or rating; paginate.
Compare	View selected products and specifications side by side.
Wishlist	Add and remove products.
Cart	Add products, change quantities, remove items, display total.
Preferences	Save preferred categories, brands, and budget.
Recommendations	Return matching catalog products with supported reasons and availability; ask clarifying questions.
Policy question	Answer from approved documents with document, section and version references.
Knowledge upload	Upload PDF, Markdown, or JSON with required metadata.
Order tracking	Retrieve the customer's simulated order and shipment events.
Combined assistance	Coordinate product, inventory, order and policy lookups.

The interface adapts to desktop and mobile screen sizes. Membership is part of the customer profile; no additional benefits are assumed.

3. Architecture and responsibilities



The backend is one application with separate shopping, AI, knowledge, orders, and security packages, keeping local execution manageable.

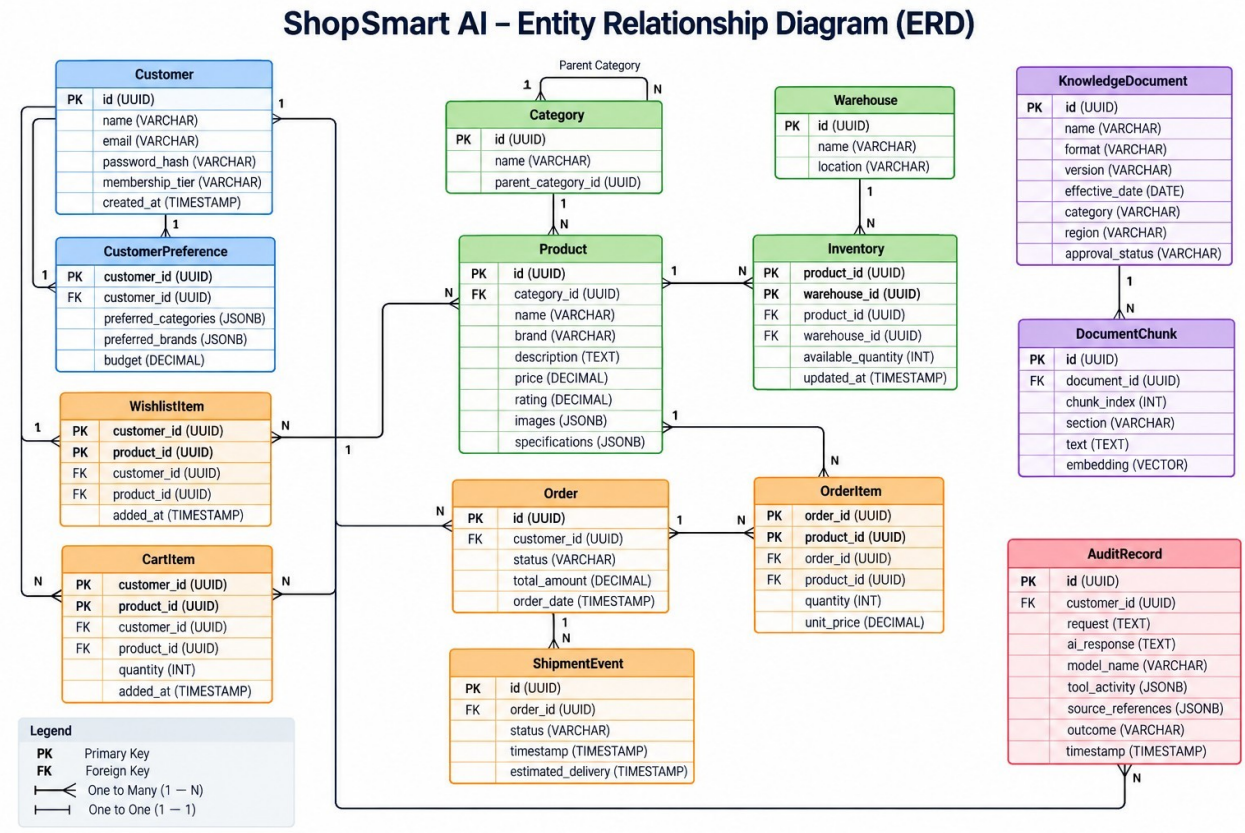
Component	Responsibility
React UI	Shopping screens, chat, document upload, order display, and source references.
Spring Boot API	Authentication, input validation, responses, and coordination.
Shopping services	Catalog, categories, comparison, wishlist, cart, and preferences.
Spring AI assistant	Interpret intent, call backend tools, and generate structured responses.
Backend tools	Read product, warehouse inventory, order, and shipment information.
RAG service	Parse, chunk, embed, retrieve evidence, and answer document questions.
PostgreSQL / pgvector	Shopping data, knowledge documents/chunks, audit records, and vector search.
Model provider	Chat generation and embeddings.

4. Data model

Entity	Minimum fields
Customer	ID, name, email, password hash, membership tier
CustomerPreference	Customer ID, preferred categories, brands, budget
Category	ID, name, parent category ID
Product	ID, name, category ID, brand, description, price, rating, images, specifications

Entity	Minimum fields
Warehouse	ID, name, location
Inventory	Product ID, warehouse ID, available quantity
WishlistItem	Customer ID, product ID
CartItem	Customer ID, product ID, quantity
Order	ID, customer ID, status, total amount, order date
OrderItem	Order ID, product ID, quantity, unit price
ShipmentEvent	ID, order ID, status, timestamp, estimated delivery
KnowledgeDocument	ID, name, format, version, effective date, category, region, approval status
DocumentChunk	ID, document ID, chunk index, section, text, embedding
AuditRecord	ID, customer ID, request, AI response, model name, tool activity, source references, outcome, timestamp

Entity Relationship Diagram



5. API CONTRACTS

All APIs use JSON except multipart document upload. Protected endpoints use the authenticated customer identity. Only APIs required for the committed case-study functionality are included.

Method and endpoint	Request	Response
POST /api/auth/login	Email, password	Session and customer profile
GET /api/categories	None	Category hierarchy
GET /api/products	Search, category, brand, price range, sort, page, size	Products and pagination
GET /api/products/{id}	Product ID	Details, specifications, availability

Method and endpoint	Request	Response
GET /api/products/compare	Product IDs	Comparison details
GET /api/preferences	None	Customer preferences
PUT /api/preferences	Categories, brands, budget	Saved preferences
GET /api/wishlist	None	Wishlist products
POST /api/wishlist	Product ID	Updated wishlist
DELETE /api/wishlist/{productId}	Product ID	Updated wishlist
GET /api/cart	None	Items and total
POST /api/cart/items	Product ID, quantity	Updated cart
PUT /api/cart/items/{productId}	Quantity	Updated cart
DELETE /api/cart/items/{productId}	Product ID	Updated cart
GET /api/orders	None	Customer orders
GET /api/orders/{id}	Order ID	Order and shipment events
GET /api/inventory/{productId}	Product ID	Warehouse quantities and availability
POST /api/documents	File, version, effective date, category, region	Document ID and ingestion status
POST /api/ai/recommendations	Message	Recommendation response
POST /api/ai/knowledge	Question, category/region context	Knowledge response
POST /api/ai/chat	Message	Response from coordinated shopping, order, or knowledge tools

6. Response formats and AI workflows

Recommendation response – source contract

```
{
  "interpretedRequirement": "Laptop for Java development and AI learning",
  "budget": 1000,
  "recommendations": [
    {
      "productId": "LAP101",
      "productName": "Dell Inspiron 15",
      "reason": "16GB RAM, Intel i7 processor, SSD storage",
      "availability": "IN_STOCK"
    }
  ],
  "clarifyingQuestion": null
}
```

Recommendation workflow

1. Read the message and saved preferences.
2. Extract category, budget, use, and specifications. Ask a question if essential details are missing.
3. Search catalog products with the required filters.
4. Check inventory and explain recommendations using actual product specifications.
5. Validate and return the structured response.

Recommendations must refer to real catalog products and respect budget constraints.

Knowledge response – source contract

```
{
  "answer": "Opened laptops can be returned within 7 days if physical damage is not present.",
  "confidence": "HIGH",
  "sources": [
    {
      "document": "Laptop Return Policy",
      "section": "Opened Products",
      "version": "2026.1"
    }
  ],
  "followUpRequired": false
}
```

The policy above is illustrative and may be returned only when supported by an uploaded, approved document.

Common error response

```
{
  "code": "INVALID_REQUEST",
  "message": "Quantity must be greater than zero."
}
```

7. Knowledge, coordination, and safeguards

How the knowledge assistant works

Upload an approved PDF, Markdown, or JSON file. The backend reads the text, splits it into small chunks, creates embeddings, and stores them with document metadata. When a customer asks a question, it finds relevant chunks and answers using only that evidence. The response cites the document, section, and version. If evidence is missing or contradictory, it says so and asks for clarification.

Confidence means strength of supporting evidence: HIGH = direct support, MEDIUM = partial support, LOW = insufficient or conflicting evidence. It is not a statistical probability.

How the assistant coordinates services

For a request such as "Where is my order, and can I cancel after shipment?", the assistant checks the customer's order status, retrieves the applicable cancellation policy, and combines the results with citations. It explains eligibility but does not cancel the order.

Basic safeguards

- Use authenticated customer identity for orders, cart, wishlist, and preferences; do not expose another customer's data.
- Store hashed passwords and keep model keys in backend environment variables.
- Only the configured demo uploader can upload documents; only approved documents are used in answers.
- Validate all API inputs and tool arguments; document text is evidence, not an instruction source.
- Show simple error messages for missing products, unavailable stock, bad files, insufficient evidence, and model timeouts.
- Keep normal shopping features usable if AI is unavailable. Save request, tools, sources, outcome, and timestamp in audit records.

8. Stage-wise implementation roadmap

The first four stages produce a complete working demonstration of the base ShopSmart AI case-study requirements. Stages five and six are optional, open-ended directions for later improvements—not commitments for the initial demo.

Stage	Implementation focus	Result
Stage 1 – Foundation and catalog	Set up React/Vite, Spring Boot, PostgreSQL/pgvector, schema, seed data, authentication, and membership profiles. Implement categories, product listings, search, filters, sorting, pagination, comparison, and responsive UI.	Working catalog and authenticated shopping UI.
Stage 2 – Shopping and order tracking	Implement preferences, wishlist, cart, quantity updates, totals, warehouse inventory, simulated orders, and shipment event display. Apply ownership checks and input validation.	Working end-to-end basic shopping and order lookup.
Stage 3 – AI product discovery	Connect chat model and Spring AI tools. Interpret shopping requests, extract budget/specifications, search the existing catalog, verify stock, return reasons and clarifying questions.	Working AI product recommendation demo.
Stage 4 – RAG and integrated assistant	Upload PDF/Markdown/JSON, chunk, embed, retrieve only approved content, display grounded citations, coordinate order + policy queries, and record audits. Test error handling and integrated UI.	Complete functional demo of all basic case-study requirements.
Stage 5 – Future enhancements (open-ended)	Potential directions: better product ranking, richer customer personalization, improved document quality checks, observability and automated testing. Scope chosen after Stage 4.	Optional improvements; not part of the base demo.
Stage 6 – Production scalability (open-ended)	Potential directions: cloud deployment, monitoring, caching, queues for ingestion, scalable vector search, stronger identity controls, and resilience. Assess load and cost before selecting architecture.	Optional production-readiness roadmap.

Scope note: payment processing, live carrier integration, stock forecasting, promotions, and automated order cancellation are not included in the four-stage base demo.